

TD N°13

Thèmes : Exception - Généricité - Package

Consigne

Rappels sur les exceptions (contrôlées ou non).

Généricité (versus polymorphisme)

Package

Exceptions

Exercice 1: Contrôlées ou non

Pour chaque phrase suivante, précisez si elle concerne les *checked* exceptions (vérifiées) et/ou les *unchecked* exceptions (non vérifiées).

- Elles doivent apparaître dans la signature des méthodes qui peuvent les lever ou les propager lors d'un appel grâce au mot-clé **throws**.
 - ☐ Les *checked* exceptions
 - ☐ Les *unchecked* exceptions
- Elles héritent directement ou indirectement de **RuntimeException** ou de **Error**.
 - ☐ Les *checked* exceptions
 - ☐ Les *unchecked* exceptions
- La déclaration de ces exceptions est optionnelle dans la signature des méthodes pouvant les lever ou les propager
 - ☐ Les *checked* exceptions
 - ☐ Les *unchecked* exception
- On peut les intercepter dans une méthode grâce à la syntaxe **try-catch**
 - ☐ Les *checked* exceptions
 - ☐ Les *unchecked* exception

Exercice 2: Méthode `Collections.min()`

La méthode `Collections.min()` est définie de la façon suivante

```
public static T min(Collection<T> coll)
Returns the minimum element of the given collection ...

Parameters:  coll - the collection whose minimum element
               isto be determined.

Returns:
    the minimum element of the given collection, according
    to the natural ordering of its elements.

Throws:
    NoSuchElementException - if the collection is empty.
```

Soit le code suivant

```
1 public class Executable {
2     static void afficheMin(List<Integer> liste) {
3         Integer minimum = Collections.min(liste);
4         System.out.println("Ma liste: " + liste);
5         System.out.println("Et voici le minimum:");
6         System.out.println(minimum);
7     }
8
9     public static void main(String[] args) {
10        List<Integer> liste = new ArrayList<>();
11        System.out.println("Liste 1");
12        afficheMin(liste);
13        System.out.println("Liste 2");
14        List<Integer> liste2 = new ArrayList<>();
15        liste2.add(1);
16        liste2.add(7);
17        liste2.add(-2);
18        afficheMin(liste2);
19    }
20 }
```

Que va-t-il se passer à la compilation ? à l'exécution ?

- En utilisant le bloc **try ... catch** dans la méthode *afficheMin()*, modifiez le code précédent pour que l'affichage soit

```
Liste1
Ma liste : []
La liste est vide
Liste2
Ma liste : [1,7,-2]
Et voici le minimum : -2
```

- Même question mais cette fois-ci en mettant le bloc **try ... catch** dans la méthode *main()*.
- Écrivez votre méthode *min()* qui lève une exception *NoSuchElementException* si la liste est vide et renvoie la valeur du minimum sinon. Quel sera la signature de cette méthode
- À l'aide des concepts précédents écrivez une méthode *max()*, puis les méthodes *produitMax()* et *sommeMax()* permettant de calculer la somme / le produit des maximums d'une liste de listes d'entiers.

```
sommeMax([[1, 3, 2], [2, 10, 1], [1, 100, 3, 2, 4]])
renverrait 113
produitMax([ [1, 3, 2], [2, 10, 1], [1, 100, 3, 2, 4] ])
renverrait 3000
```

Exercice 3: Retrait bancaire (bonus)

Lorsque qu'un client d'une banque souhaite retirer de l'argent, il se présente à un guichet, indique son numéro de compte et le montant qu'il souhaite obtenir.

Après avoir renseigné le numéro de compte et le montant, une validation du compte est faite auprès du système central, ainsi qu'une vérification que le compte est bien approvisionné,

Le débit est effectué et la somme demandée est fournie au client.

Il s'agit d'un scénario nominal.

- Proposez plusieurs scénarios exceptionnels qui pourraient se produire.
- En utilisant des Exceptions, proposez une méthode *debit()* de la classes **Compte** permettant de gérer vos scénarios.
- Le Système central vérifie la validation du numéro de compte et la vérification du solde. Proposez des codes pour les méthodes *validationCompte()* et *verificationSolde()*.

Généricité

Exercice 4: Simple

On vous demande d'implémenter une classe générique **Simple** possédant un seul attribut, deux constructeurs (l'un sans paramètre, l'autre avec un paramètre du type de l'attribut), un getter et un setter.

Vous testerez cette classe dans un programme principal avec un *Integer* (12), un *String* ("toto") et un *Double* (5.5).

Est-il possible de déclarer un Simple de Simple d'entier ? comment ? et un Simple de Simple d'un Couple de chaines ? (testez)

Exercice 5: Comparable<T>

L'interface **Comparable<T>** permet de définir des objets *naturellement comparables*, elle est utilisée par la méthode sort de Collections.

- Écrivez une classe **CoupleEntiers** qui soit naturellement triable par premier élément croissant et dans le cas d'égalité, compare par second élément.
- Écrivez une classe **CoupleDouble** qui soit naturellement triable par premier élément croissant.
- Définissez un exécutable.
- Écrivez une classe générique **Couple<T extends Comparable< ? super T>>** qui soit naturellement triable par premier élément croissant.

Remarquez qu'il suffirait pour la classe T d'être comparable avec n'importe lequel de ses ancêtres dans la hiérarchie, c'est-à-dire que si par exemple T étend une classe qui est Comparable <un des parents de T> alors T peut être comparé avec T.

On peut alors modifier la classe pour avoir le profil :

Couple<T extends Comparable< ? super T>> .

- Testez cette classe dans un exécutable, sur des entiers et des doubles ainsi que sur des couples d'entiers ou de doubles.

Exercice 6: Ajoutable (bonus)

- Écrivez une interface **Ajoutable**<**T**> contenant une méthode $T \text{ ajoute}(T \ x)$. Cette méthode, lorsqu'elle sera implémentée, correspondra à la notion de $+$.

Par exemple si on a une classe `Entier` implémentant `Ajoutable<Entier>`, `cinq.ajoute(trois)` renverrait huit.

- Écrivez une classe **Entier** implémentant `Ajoutable`.
- Écrivez une classe **Ajoutables(avec un s)** contenant une méthode `somme()` prenant en entrée une liste d'`Ajoutable` et renvoyant la somme de ces `Ajoutables`. Essayer de trouver le profil de `somme()`.
- Utilisez la méthode précédente pour faire la somme des éléments d'un tableau d'`Entiers`, dans un exécutable.
- Utilisez la méthode précédente pour pouvoir faire la concaténation de chaînes de caractères. Que faut-il faire?